

Knowledge Representation

Lecture 3:

Reasoning about Actions III

dr Anna Maria Radzikowska

Wydział Matematyki i Nauk Informacyjnych

Pokój 504, gmach MiNI

E-mail: A.Radzikowska@mini.pw.edu.pl

Action Language $\mathcal{AR}$ – recall...

Statements:

- *value statement:* α **after** $A_1, \dots, A_n$
- *observation statement:* **observable** α **after** $A_1, \dots, A_n$
- *effect statement:* A **causes** α **if** π
- *release statement:* A **releases** f **if** π
- *constraint statement:* **always** α .

Action Language $\mathcal{AR}$ – recall...

Statements:

- *value statement:* α **after** $A_1, \dots, A_n$
- *observation statement:* **observable** α **after** $A_1, \dots, A_n$
- *effect statement:* A **causes** α **if** π
- *release statement:* A **releases** f **if** π
- *constraint statement:* **always** α .

A **structure** for a language $\mathcal{L}$ of the class $\mathcal{AR}$ is a triple $S = (\Sigma, \sigma_0, Res)$, where

- Σ is a set of states
- $\sigma_0 \in \Sigma$ is the initial state
- $Res : \mathcal{Ac} \times \Sigma \rightarrow 2^\Sigma$ is a transition function.

Action Language $\mathcal{AR}$ – model

A structure $S = (\Sigma, \sigma_0, Res)$ is a **model of** an action domain D iff

- (M.1) Σ is the set of all states for D
- (M.2) every value statement and every observation statement is true in S
- (M.3) for any $A \in \mathcal{Ac}$ and for any $\sigma \in \Sigma$, $Res(A, \sigma)$ is the set of all states $\sigma' \in \Sigma$ for which the sets $New(A, \sigma, \sigma')$ are minimal (wrt set inclusion).

Recall that $New(A, \sigma, \sigma')$ is the set of literals $\bar{f}$ that hold in σ' and

- f is inertial and $\sigma(f) \neq \sigma'(f)$, or
- there is a statement A **releases** f **if** π in D such that $\sigma \models \pi$.

Queries in $\mathcal{AR}$

By a *query of $\mathcal{AR}$* we mean any expression of the form:

• *value queries:*

necessary α after $A_1, \dots, A_n$ from π

possibly α after $A_1, \dots, A_n$ from π

The 1st query states that α **always** holds after performing the sequence $A_1, \dots, A_n$ of actions from any state satisfying π .

The 2nd query says that α **sometimes** holds after executing $A_1, \dots, A_n$ from any state satisfying π .

When the option **from** π is omitted, these queries refer to the initial state.

Basic queries (cont.)

• *executability query:*

executable $A_1, \dots, A_n$ *from* π

Intuitively, this query states that the sequence $(A_1, \dots, A_n)$ is executable from any state satisfying π .

Again, if the phrase *from* π is omitted, then the initial state is to be taken into account.

Basic queries (cont.)

• *accessibility query:*

accessible γ from π .

This query states that the goal γ is achieved from any state satisfying π .

If the phrase *from* π is omitted, then it refers to the initial state.

Basic queries (cont.)

• *accessibility query:*

accessible γ from π .

This query states that the goal γ is achieved from any state satisfying π .

If the phrase *from* π is omitted, then it refers to the initial state.

Underlying intuition:

There exists a sequence $(A_1, \dots, A_n)$, $n \geq 0$, of actions, executable from any state satisfying π (resp. the initial state), which leads to states satisfying γ . Various sequences of actions, starting in various initial states, may lead to various states, where γ holds.

Basic queries (cont.)

• *goal query:*

goal γ from π .

Intuitively, this query returns the shortest sequence $(A_1, \dots, A_n)$ of actions, executable from any state satisfying π (if such a sequence exists at all), which leads to where the goal condition γ holds.

When the goal is to be achieved from the initial state, the phrase ***from*** π will be omitted.

Satisfiability

Let D be an action domain and let Q be a query of $\mathcal{AR}$. We say that Q is a *consequence of D* , in symbols $D \models Q$, iff

- if Q is a value query of the form

necessary α after $A_1, \dots, A_n$ from π ,

then $D \models Q$ iff for every model $S = (\Sigma, \sigma_0, Res)$ of D , for every state $\sigma \in \Sigma$, and for **every** mapping Ψ_S , if $\sigma \models \pi$ then $\Psi_S((A_1, \dots, A_n), \sigma) \models \alpha$

Satisfiability

Let D be an action domain and let Q be a query of $\mathcal{AR}$. We say that Q is a **consequence of D** , in symbols $D \models Q$, iff

- if Q is a value query of the form

necessary α after $A_1, \dots, A_n$ from π ,

then $D \models Q$ iff for every model $S = (\Sigma, \sigma_0, Res)$ of D , for every state $\sigma \in \Sigma$, and for **every** mapping Ψ_S , if $\sigma \models \pi$ then $\Psi_S((A_1, \dots, A_n), \sigma) \models \alpha$

- if Q is a value query

necessary α after $A_1, \dots, A_n$,

then $D \models Q$ iff for every model $S = (\Sigma, \sigma_0, Res)$ of D , and for **every** mapping Ψ_S , it holds $\Psi_S((A_1, \dots, A_n), \sigma_0) \models \alpha$.

Satisfiability (cont.)

- if Q is a value query of the form

possibly α after $A_1, \dots, A_n$ from π ,

then $D \models Q$ iff for every model $S = (\Sigma, \sigma_0, Res)$ of D , for every state $\sigma \in \Sigma$ and for **some** mapping Ψ_S , if $\sigma \models \pi$ then $\Psi_S((A_1, \dots, A_n), \sigma) \models \alpha$.

Satisfiability (cont.)

- if Q is a value query of the form

possibly α after $A_1, \dots, A_n$ from π ,

then $D \models Q$ iff for every model $S = (\Sigma, \sigma_0, Res)$ of D , for every state $\sigma \in \Sigma$ and for **some** mapping Ψ_S , if $\sigma \models \pi$ then $\Psi_S((A_1, \dots, A_n), \sigma) \models \alpha$.

- if Q is a value query of the form

possibly α after $A_1, \dots, A_n$,

then $D \models Q$ iff for every model $S = (\Sigma, \sigma_0, Res)$ of D , and for **some** mapping Ψ_S , it holds $\Psi_S((A_1, \dots, A_n), \sigma_0) \models \alpha$.

Example: Tossing a coin

For example, let an action domain D be as follows:

initially heads

Toss *releases heads*.

Then

$D \models \text{possibly } \neg \text{heads after Toss}$

$D \models \text{possibly heads after Toss}$

$D \not\models \text{necessary ; } \neg \text{head after Toss}$

$D \not\models \text{necessary head after Toss.}$

Satisfiability (cont.)

- if Q is an executability query of the form

executable $A_1, \dots, A_n$ ***from*** π ,

then $D \models Q$ iff for every model $S = (\Sigma, \sigma_0, Res)$ of D , for every state $\sigma \in \Sigma$, and for every mapping Ψ_S , if $\sigma \models \pi$ then $\Psi_S((A_1, \dots, A_n), \sigma)$ is defined.

Satisfiability (cont.)

- if Q is an executability query of the form

executable $A_1, \dots, A_n$ ***from*** π ,

then $D \models Q$ iff for every model $S = (\Sigma, \sigma_0, Res)$ of D , for every state $\sigma \in \Sigma$, and for every mapping Ψ_S , if $\sigma \models \pi$ then $\Psi_S((A_1, \dots, A_n), \sigma)$ is defined.

- if Q is an executability query of the form

executable $A_1, \dots, A_n$,

then $D \models Q$ iff for every model $S = (\Sigma, \sigma_0, Res)$ of D and for every mapping Ψ_S , $\Psi_S((A_1, \dots, A_n), \sigma_0)$ is defined.

Satisfiability (cont.)

- if Q is an accessibility query of the form

accessible γ from π ,

then $D \models Q$ iff for every model $S = (\Sigma, \sigma_0, Res)$ of D , for every state $\sigma \in \Sigma$ and for every mapping Ψ_S , if $\sigma \models \pi$ then there exists a sequence $(A_1, \dots, A_n)$ of actions such that

- $\Psi_S((A_1, \dots, A_n), \sigma)$ is defined
- $\Psi_S((A_1, \dots, A_n), \sigma) \models \gamma$

Satisfiability (cont.)

- if Q is an accessibility query of the form

accessible γ ,

then $D \models Q$ iff for every model $S = (\Sigma, \sigma_0, Res)$ of D , and for every mapping Ψ_S , there exists a sequence $(A_1, \dots, A_n)$ of actions, $n \geq 0$, such that

- $\Psi_S((A_1, \dots, A_n), \sigma_0)$ is defined
- $\Psi_S((A_1, \dots, A_n), \sigma_0) \models \gamma$.

An example

Consider an action domain:

A causes $p \vee q$;

B causes $\neg r$;

impossible B if $\neg q$.

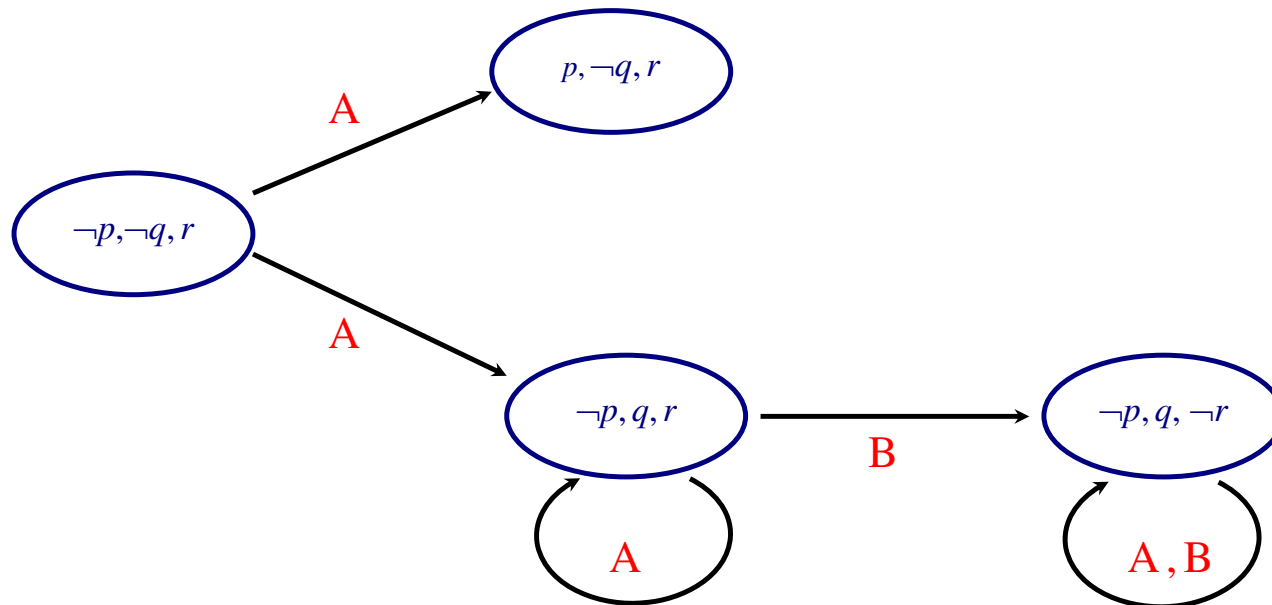
An example

Consider an action domain:

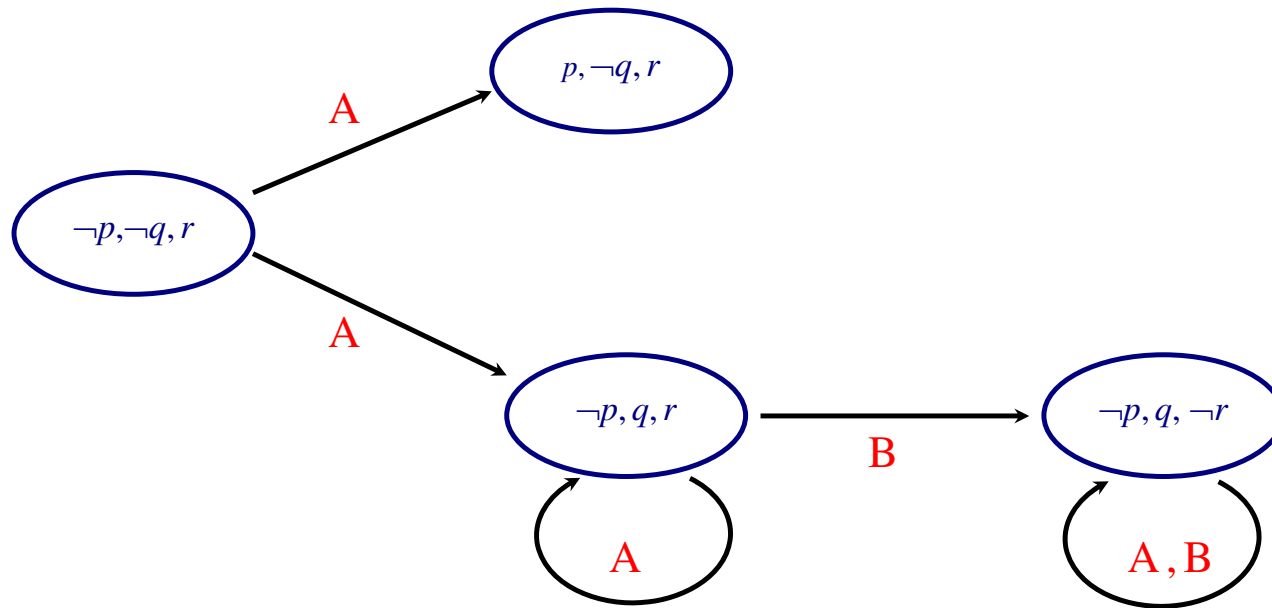
A causes $p \vee q$;

B causes $\neg r$;

impossible B if $\neg q$.

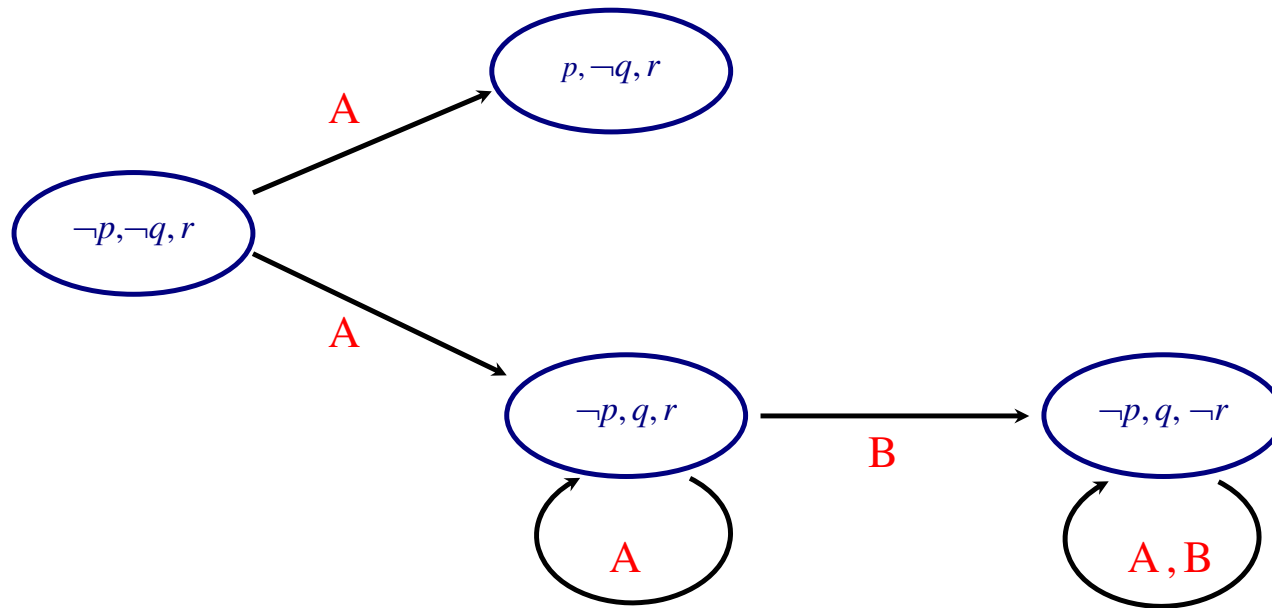


An example (cont.)



Query: *accessible* $\neg r$ *from* $\neg p$.

An example (cont.)

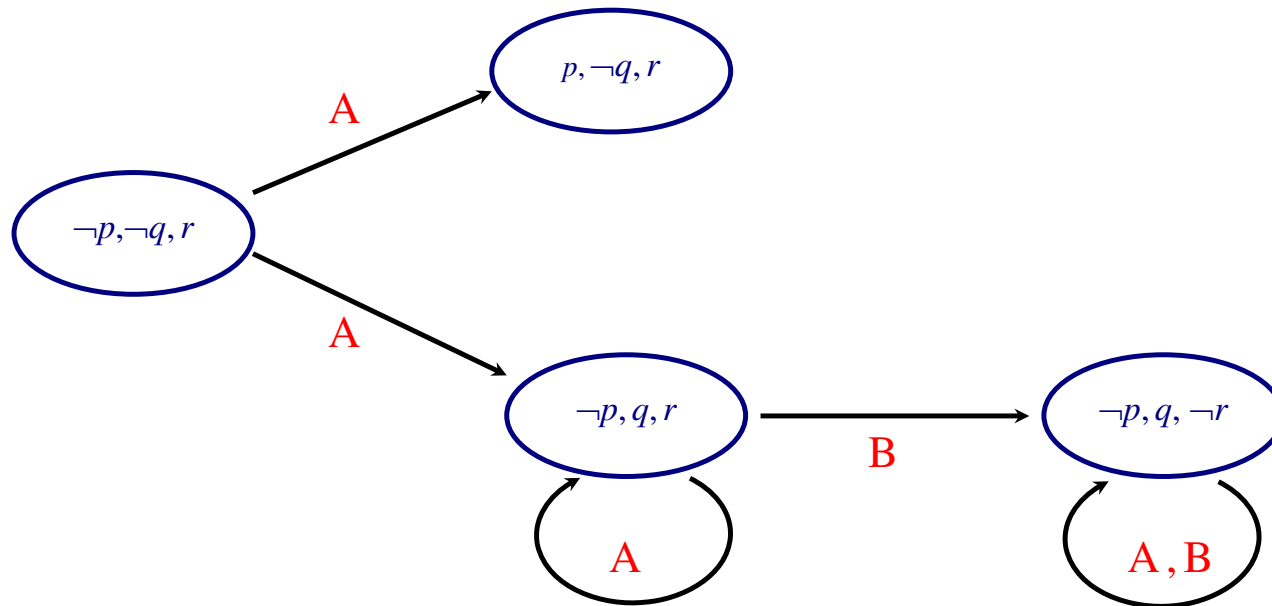


Query: *accessible* $\neg r$ from $\neg p$.

$D \not\models$ *accessible* $\neg r$ from $\neg p$.

WHY?

An example (cont.)



Query: *accessible* $\neg r$ *from* $\neg p$.

$D \not\models$ *accessible* $\neg r$ *from* $\neg p$.

WHY? Because performing the sequence (A, B) in the state $(\neg p, \neg q, r)$ may (but not need) lead to the state $(\neg p, q, \neg r)$.

Satisfiability (cont.)

- if Q is a goal query of the form

goal γ from π ,

then $D \models Q$ iff there exists a sequence $(A_1, \dots, A_n)$ of actions such that for every model $S = (\Sigma, \sigma_0, Res)$ of D , for every state $\sigma \in \Sigma$, and for every mapping Ψ_S , if $\sigma \models \pi$ then

- $\Psi_S((A_1, \dots, A_n), \sigma)$ is defined
- $\Psi_S((A_1, \dots, A_n), \sigma) \models \gamma$.

The sequence $(A_1, \dots, A_n)$, if it exists, is called an ***answer for the query Q wrt D .***

Satisfiability (cont.)

- if Q is a goal query of the form

goal γ

then $D \models Q$ iff there exists a sequence $(A_1, \dots, A_n)$ of actions such that for every model $S = (\Sigma, \sigma_0, Res)$ of D and for every mapping Ψ_S , it holds

- $\Psi_S((A_1, \dots, A_n), \sigma_0)$ is defined
- $\Psi_S((A_1, \dots, A_n), \sigma_0) \models \gamma$.

As before, the sequence $(A_1, \dots, A_n)$, if it exists, is called an *answer for the query Q wrt D* .

Actions with Qualifications

Qualification problem – recall

*The qualification problem concerns conditions under which the performance of an action is possible, it proceeds successfully and leads to expected results. Since all preconditions of actions are usually immense, it is usually unreasonable (if ever possible) to explicitly enumerate and check **all** of possibilities (even most unlikely).*

Introductory example (Lin and Reiter, 1994)

Consider an action domain :

In an ancient kingdom there are two blocks, A and B. Either block may be painted yellow, but by order of the emperor at most one of the blocks is permitted to be yellow at a time. Initially the first block is yellow. A painter tried to paint the second one yellow.

Introductory example (Lin and Reiter, 1994)

Consider an action domain :

In an ancient kingdom there are two blocks, A and B. Either block may be painted yellow, but by order of the emperor at most one of the blocks is permitted to be yellow at a time. Initially the first block is yellow. A painter tried to paint the second one yellow.

Representation in $\mathcal{AR}$:

always $\neg \text{yellow}A \vee \neg \text{yellow}B$;

initially $\text{yellow}A$;

$\text{PAINT}A$ ***causes*** $\text{yellow}A$;

$\text{PAINT}B$ ***causes*** $\text{yellow}B$.

Introductory example (cont.)

Here $\Sigma = \{\sigma_0, \sigma_1, \sigma_2\}$, where

$$\begin{aligned}\sigma_0 &= \{yellowA, \neg yellowB\} & \sigma_1 &= \{\neg yellowA, \neg yellowB\}. \\ \sigma_2 &= \{\neg yellowA, yellowB\}\end{aligned}$$

By simple calculations we get:

$$Res_0(PAINTB, \sigma_0) = Res(PAINTB, \sigma_0) = \{\sigma_2\}.$$

So painting the block B changes the color of the block A!

Example 2: Enticing Fred

Consider an action domain:

***initially** loaded $\wedge$ walking;*

***always** walking $\rightarrow$ alive;*

SHOOT ***causes** $\neg$ loaded;*

SHOOT ***causes** $\neg$ alive **if** loaded;*

ENTICE ***causes** walking.*

Example 2: Enticing Fred

Consider an action domain:

initially $loaded \wedge walking$;
always $walking \rightarrow alive$;
SHOOT *causes* $\neg loaded$;
SHOOT *causes* $\neg alive$ *if* $loaded$;
ENTICE *causes* $walking$.

Let

$$\sigma_0 = \{alive, loaded, walking\}$$
$$\sigma_1 = \{\neg alive, \neg loaded, \neg walking\}.$$

It is easy to check that $\boxed{Res(\text{SHOOT}, \sigma_0) = \{\sigma_1\}}$.

Example 2 (cont.)

Furthermore, let

$$\sigma_2 = \{\neg alive, loaded, \neg walking\}$$

$$\sigma_3 = \{alive, \neg loaded, walking\}$$

$$\sigma_4 = \{alive, loaded, walking\}.$$

In $\mathcal{AR}$ we get:

$$Res_0(\text{ENTICE}, \sigma_2) = \{\sigma_3, \sigma_4\}$$

$$New(\text{ENTICE}, \sigma_2, \sigma_3) = \{alive, \neg loaded, walking\}$$

$$* \quad New(\text{ENTICE}, \sigma_2, \sigma_4) = \{alive, walking\}$$

$$\underline{Res(\text{ENTICE}, \sigma_2) = \{\sigma_4\}}.$$

In other words, **enticing a dead turkey makes him miraculously alive !!!**

Example 2 (cont.)

Furthermore, let

$$\sigma_2 = \{\neg alive, loaded, \neg walking\}$$

$$\sigma_3 = \{alive, \neg loaded, walking\}$$

$$\sigma_4 = \{alive, loaded, walking\}.$$

In $\mathcal{AR}$ we get:

$$Res_0(\text{ENTICE}, \sigma_2) = \{\sigma_3, \sigma_4\}$$

$$New(\text{ENTICE}, \sigma_2, \sigma_3) = \{alive, \neg loaded, walking\}$$

$$* \quad New(\text{ENTICE}, \sigma_2, \sigma_4) = \{alive, walking\}$$

$$\underline{Res(\text{ENTICE}, \sigma_2) = \{\sigma_4\}}.$$

In other words, **enticing a dead turkey makes him miraculously alive !!!**

The reasoning methods of $\mathcal{AR}$ is not adequate to represent this scenario.

Basic assumptions

- Inertia law
- Complete information about all actions and all fluents
- Nondeterminism is allowed.
- Whenever any indirect precondition of an action fails, the action is unexecutable
- Action can be unexecutable in some states.

Basic assumptions

- Inertia law
- Complete information about all actions and all fluents
- Nondeterminism is allowed.
- Whenever any indirect precondition of an action fails, the action is unexecutable
- Action can be unexecutable in some states.

To represent this class of dynamic systems we will use action languages of the class *AQ*.

Action Language AQ

Syntax is identical as in AR .

Semantics of AQ

Definition 3.1. Let D be an action domain. A **structure** for a language $\mathcal{L}$ of the class AQ is a triple $S = (\Sigma, \sigma_0, Res)$ such that

- Σ is a set of states
- $\sigma_0 \in \Sigma$ is the initial state
- $Res : \mathcal{A}c \times \Sigma \rightarrow 2^\Sigma$ is a transition function.

Semantics of AQ

Denote

- D^- – the action domain obtained from D by removing all constraint statements.

Semantics of AQ

Denote

- D^- – the action domain obtained from D by removing all constraint statements.
- $Mod(D^-)$ – the set of all models $S^- = (\Sigma^-, \sigma_0^-, Res^-)$ of D^- in the sense of $\mathcal{AR}$.

Semantics of AQ

Denote

- D^- – the action domain obtained from D by removing all constraint statements.
- $Mod(D^-)$ – the set of all models $S^- = (\Sigma^-, \sigma_0^-, Res^-)$ of D^- in the sense of $\mathcal{AR}$.
- $Mod^*(D^-)$ – the set of all models $S = (\Sigma, \sigma_0, Res) \in Mod(D^-)$ such that σ_0 satisfies all constraint statements in D .

Semantics of AQ (cont.)

Let $S^* = (\Sigma^*, \sigma_0^*, Res^*) \in Mod^*(D^-)$. Put

- $\Sigma \subseteq \Sigma^*$ be the set of all states satisfying all constraint statements in D
- for any action $A \in \mathcal{Ac}$ and for any state $\sigma \in \Sigma$,
put $\boxed{Res(A, \sigma) = Res^*(A, \sigma) \cap \Sigma}$.

Semantics of $\mathcal{AQ}$ (cont.)

Let $S^* = (\Sigma^*, \sigma_0^*, Res^*) \in Mod^*(D^-)$. Put

- $\Sigma \subseteq \Sigma^*$ be the set of all states satisfying all constraint statements in D
- for any action $A \in \mathcal{Ac}$ and for any state $\sigma \in \Sigma$,
put $\boxed{Res(A, \sigma) = Res^*(A, \sigma) \cap \Sigma}$.

Definition 3.2. The structure $S = (\Sigma, \sigma_0, Res)$ defined as above is a *model of* D .

Introductory example (cont.)

***always** $\neg yellowA \vee \neg yellowB$;*

***initially** $yellowA$;*

$PAINTA$ ***causes** $yellowA$;*

$PAINTB$ ***causes** $yellowB$.*

Now $\Sigma = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3\}$, where

$\sigma_0 = \{yellowA, \neg yellowB\}$ $\sigma_1 = \{\neg yellowA, \neg yellowB\}$

$\sigma_2 = \{\neg yellowA, yellowB\}$ $\sigma_3 = \{yellowA, yellowB\}$.

Introductory example (cont.)

$$\begin{aligned}\sigma_0 &= \{yellowA, \neg yellowB\} & \sigma_1 &= \{\neg yellowA, \neg yellowB\} \\ \sigma_2 &= \{\neg yellowA, yellowB\} & \sigma_3 &= \{yellowA, yellowB\}.\end{aligned}$$

$$Res_0^-(PAINTB, \sigma_0) = \{\sigma_2, \sigma_3\}$$

$$New(PAINTB, \sigma_0, \sigma_2) = \{\neg yellowA, yellowB\}$$

$$* \quad New(PAINTB, \sigma_0, \sigma_3) = \{yellowB\}$$

$$Res^-(PAINTB, \sigma_0) = \{\sigma_3\}$$

$$\boxed{Res(PAINTB, \sigma_0) = \emptyset.}$$

Introductory example (cont.)

$$\begin{aligned}\sigma_0 &= \{yellowA, \neg yellowB\} & \sigma_1 &= \{\neg yellowA, \neg yellowB\} \\ \sigma_2 &= \{\neg yellowA, yellowB\} & \sigma_3 &= \{yellowA, yellowB\}.\end{aligned}$$

Analogously:

$$Res_0^-(\text{PAINTA}, \sigma_2) = \{\sigma_0, \sigma_3\}$$

$$New(\text{PAINTA}, \sigma_2, \sigma_0) = \{yellowA, \neg yellowB\}$$

$$* \quad New(\text{PAINTA}, \sigma_2, \sigma_3) = \{yellowA\}$$

$$Res^-(\text{PAINTA}, \sigma_2) = \{\sigma_3\}$$

$$\boxed{Res(\text{PAINTA}, \sigma_2) = \emptyset.}$$

Introductory example (cont.)

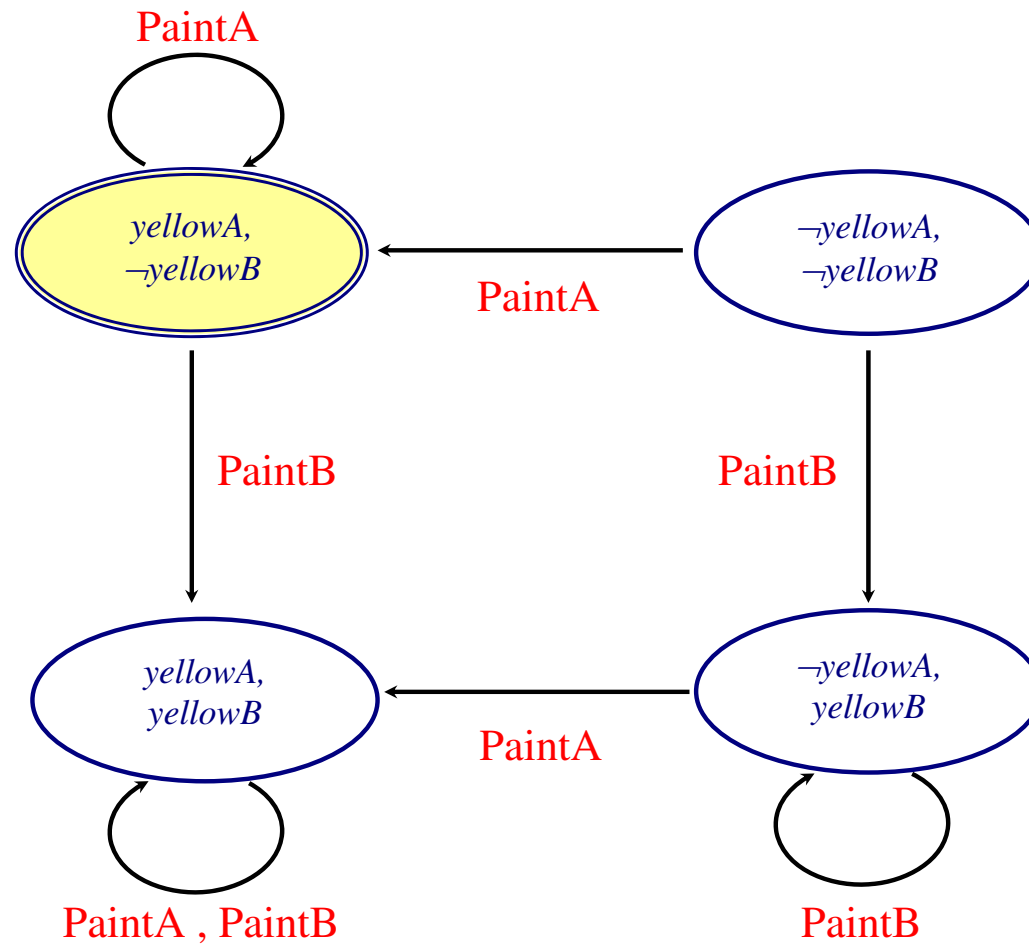


Fig.3.1 : Model of D^-

Introductory example (cont.)

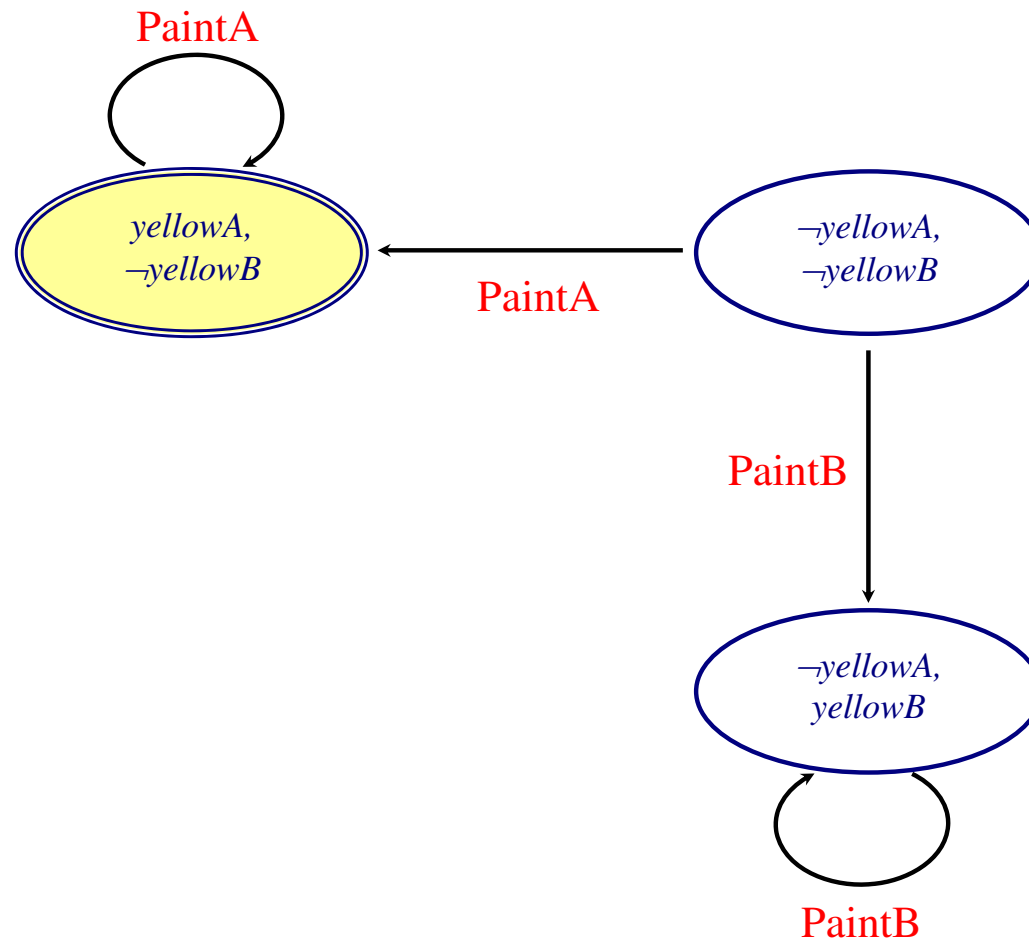


Fig.3.2 : Model of D

Example 2: Enticing Fred (cont.)

$$\sigma_0 = \{alive, loaded, walking\}$$

$$\sigma_1 = \{alive, \neg loaded, walking\}$$

$$\sigma_2 = \{\neg alive, loaded, walking\}$$

$$\sigma_3 = \{\neg alive, \neg loaded, walking\}$$

$$\sigma_4 = \{alive, loaded, \neg walking\}$$

$$\sigma_5 = \{alive, \neg loaded, \neg walking\}$$

$$\sigma_6 = \{\neg alive, loaded, \neg walking\}$$

$$\sigma_7 = \{\neg alive, \neg loaded, \neg walking\}.$$

$$Res_0^-(ENTICE, \sigma_6) = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3\}$$

$$New(ENTICE, \sigma_6, \sigma_0) = \{alive, walking\}$$

$$New(ENTICE, \sigma_6, \sigma_1) = \{alive, \neg loaded, walking\}$$

$$\star \quad New(ENTICE, \sigma_6, \sigma_2) = \{walking\}$$

$$New(ENTICE, \sigma_6, \sigma_3) = \{\neg loaded, walking\}$$

$$Res^-(ENTICE, \sigma_6,) = \{\sigma_2\}$$

$$\boxed{Res(ENTICE, \sigma_6) = \emptyset}.$$

Example 2: Enticing Fred (cont.)

$$\sigma_0 = \{alive, loaded, walking\}$$

$$\sigma_1 = \{alive, \neg loaded, walking\}$$

$$\sigma_2 = \{\neg alive, loaded, walking\}$$

$$\sigma_3 = \{\neg alive, \neg loaded, walking\}$$

$$\sigma_4 = \{alive, loaded, \neg walking\}$$

$$\sigma_5 = \{alive, \neg loaded, \neg walking\}$$

$$\sigma_6 = \{\neg alive, loaded, \neg walking\}$$

$$\sigma_7 = \{\neg alive, \neg loaded, \neg walking\}.$$

Now consider the shooting action in the initial state σ_0 .

Example 2: Enticing Fred (cont.)

$$\sigma_0 = \{alive, loaded, walking\}$$

$$\sigma_1 = \{alive, \neg loaded, walking\}$$

$$\sigma_2 = \{\neg alive, loaded, walking\}$$

$$\sigma_3 = \{\neg alive, \neg loaded, walking\}$$

$$\sigma_4 = \{alive, loaded, \neg walking\}$$

$$\sigma_5 = \{alive, \neg loaded, \neg walking\}$$

$$\sigma_6 = \{\neg alive, loaded, \neg walking\}$$

$$\sigma_7 = \{\neg alive, \neg loaded, \neg walking\}.$$

Now consider the shooting action in the initial state σ_0 .

$$Res_0^-(SHOOT, \sigma_0) = \{\sigma_3, \sigma_7\}$$

$$\star \quad New(SHOOT, \sigma_0, \sigma_3) = \{\neg alive, \neg loaded\}$$

$$New(SHOOT, \sigma_0, \sigma_7) = \{\neg alive, \neg loaded, \neg walking\}$$

$$Res^-(SHOOT, \sigma_0) = \{\sigma_3\}$$

$$\boxed{Res(SHOOT, \sigma_0) = \emptyset}.$$

Example 2: Enticing Fred (cont.)

$$\sigma_0 = \{alive, loaded, walking\}$$

$$\sigma_1 = \{alive, \neg loaded, walking\}$$

$$\sigma_2 = \{\neg alive, loaded, walking\}$$

$$\sigma_3 = \{\neg alive, \neg loaded, walking\}$$

$$\sigma_4 = \{alive, loaded, \neg walking\}$$

$$\sigma_5 = \{alive, \neg loaded, \neg walking\}$$

$$\sigma_6 = \{\neg alive, loaded, \neg walking\}$$

$$\sigma_7 = \{\neg alive, \neg loaded, \neg walking\}.$$

Now consider the shooting action in the initial state σ_0 .

$$Res_0^-(SHOOT, \sigma_0) = \{\sigma_3, \sigma_7\}$$

$$\star \quad New(SHOOT, \sigma_0, \sigma_3) = \{\neg alive, \neg loaded\}$$

$$New(SHOOT, \sigma_0, \sigma_7) = \{\neg alive, \neg loaded, \neg walking\}$$

$$Res^-(SHOOT, \sigma_0) = \{\sigma_3\}$$

$$\boxed{Res(SHOOT, \sigma_0) = \emptyset}.$$

So it is impossible to shoot a walking turkey!

Problem

Therefore the following question arises:

*Should we reject inadmissible states **before** or **after** minimization of changes?*

Problem

Therefore the following question arises:

*Should we reject inadmissible states **before** or **after** minimization of changes?*

Using some simplification we can give the following answer:

- If constraints influence only on actions' **ramifications** and do not influence on actions' qualifications, then inadmissible states are to be rejected **before** minimization of changes.

Problem

Therefore the following question arises:

*Should we reject inadmissible states **before** or **after** minimization of changes?*

Using some simplification we can give the following answer:

- If constraints influence only on actions' **ramifications** and do not influence on actions' qualifications, then inadmissible states are to be rejected **before** minimization of changes.
- If constraints influence only on actions' **qualificatins** and do not influence on actions' ramfications, then inadmissible states should be rejected **after** minimization of changes.

However,

What should be done if a constraint works for ramifications wrt one action, and at the same time for qualifications wrt another one?

However,

What should be done if a constraint works for ramifications wrt one action, and at the same time for qualifications wrt another one?

Syntax of ARQ

The set of statements of AR is extended by an **fluent preserving statement**:

A preserves f if π

Intuitively, in any state satisfying π , the action A , when performed, does not change the value of the fluent f .

Semantics of $\mathcal{ARQ}$

A **structure** for a language $\mathcal{L}$ of the class $\mathcal{ARQ}$ is a triple $S = (\Sigma, \sigma_0, Res)$ such that

- Σ is a set of states
- $\sigma_0 \in \Sigma$ is the initial state, and
- $Res : \Sigma \times \mathcal{Ac} \rightarrow 2^\Sigma$ is a transition function.

Semantics of $\mathcal{ARQ}$

A **structure** for a language $\mathcal{L}$ of the class $\mathcal{ARQ}$ is a triple $S = (\Sigma, \sigma_0, Res)$ such that

- Σ is a set of states
- $\sigma_0 \in \Sigma$ is the initial state, and
- $Res : \Sigma \times \mathcal{Ac} \rightarrow 2^\Sigma$ is a transition function.

Let D^- be the action domain obtained from D by removing all fluent preserving statements and let $Mod(D^-)$ be the set of models of D^- in the sense of $\mathcal{AR}$.

Semantics of $\mathcal{ARQ}$ – models

Definition 3.3. A structure $S = (\Sigma, \sigma_0, Res) \in Mod(D^-)$ is a **model of D** iff for any action $A \in \mathcal{Ac}$ and for any state $\sigma \in \Sigma$,

$$Res(A, \sigma) = \{\sigma' \in \Sigma : (A \text{ **preserves** } f \text{ **if** } \pi) \in D \wedge \sigma \models \pi \implies (\sigma(f) = \sigma'(f))\}.$$

Example 2: Enticing Fred (cont.)

initially loaded $\wedge$ walking;
always walking $\rightarrow$ alive;
SHOOT **causes** $\neg$ loaded;
SHOOT **causes** $\neg$ alive **if** loaded;
ENTICE **causes** walking
ENTICE **preserves** alive.

All admissible states:

$$\begin{array}{ll} \sigma_0 = \{alive, loaded, walking\} & \sigma_3 = \{alive, \neg loaded, walking\} \\ \sigma_1 = \{alive, loaded, \neg walking\} & \sigma_4 = \{alive, \neg loaded, \neg walking\} \\ \sigma_2 = \{\neg alive, loaded, \neg walking\} & \sigma_5 = \{\neg alive, \neg loaded, \neg walking\}. \end{array}$$

Example 2: Enticing Fred (cont.)

$$\begin{aligned}\sigma_0 &= \{alive, loaded, walking\} & \sigma_3 &= \{alive, \neg loaded, walking\} \\ \sigma_1 &= \{alive, loaded, \neg walking\} & \sigma_4 &= \{alive, \neg loaded, \neg walking\} \\ \sigma_2 &= \{\neg alive, loaded, \neg walking\} & \sigma_5 &= \{\neg alive, \neg loaded, \neg walking\}.\end{aligned}$$

$$Res_0^-(ENTICE, \sigma_2) = \{\sigma_0, \sigma_3\}$$

$$\star \quad New(ENTICE, \sigma_2, \sigma_0) = \{alive, walking\}$$

$$New(ENTICE, \sigma_2, \sigma_3) = \{alive, \neg loaded, walking, \}$$

$$Res^-(ENTICE, \sigma_2) = \{\sigma_0\}.$$

Example 2: Enticing Fred (cont.)

$$\begin{aligned}\sigma_0 &= \{alive, loaded, walking\} & \sigma_3 &= \{alive, \neg loaded, walking\} \\ \sigma_1 &= \{alive, loaded, \neg walking\} & \sigma_4 &= \{alive, \neg loaded, \neg walking\} \\ \sigma_2 &= \{\neg alive, loaded, \neg walking\} & \sigma_5 &= \{\neg alive, \neg loaded, \neg walking\}.\end{aligned}$$

$$Res_0^-(ENTICE, \sigma_2) = \{\sigma_0, \sigma_3\}$$

$$\star \quad New(ENTICE, \sigma_2, \sigma_0) = \{alive, walking\}$$

$$New(ENTICE, \sigma_2, \sigma_3) = \{alive, \neg loaded, walking, \}$$

$$Res^-(ENTICE, \sigma_2) = \{\sigma_0\}.$$

Since $\sigma_0(walking) \neq \sigma_2(walking)$, we get $\sigma_0 \notin Res(ENTICE, \sigma_2)$.

Hence $\boxed{Res(ENTICE, \sigma_2) = \emptyset}$.

Example 2: Enticing Fred (cont.)

$$\begin{aligned}\sigma_0 &= \{alive, loaded, walking\} & \sigma_3 &= \{alive, \neg loaded, walking\} \\ \sigma_1 &= \{alive, loaded, \neg walking\} & \sigma_4 &= \{alive, \neg loaded, \neg walking\} \\ \sigma_2 &= \{\neg alive, loaded, \neg walking\} & \sigma_5 &= \{\neg alive, \neg loaded, \neg walking\}.\end{aligned}$$

$$Res_0^-(\text{SHOOT}, \sigma_0) = Res^-(\text{SHOOT}, \sigma_0) = \{\sigma_5\}.$$

Example 2: Enticing Fred (cont.)

$$\begin{aligned}\sigma_0 &= \{alive, loaded, walking\} & \sigma_3 &= \{alive, \neg loaded, walking\} \\ \sigma_1 &= \{alive, loaded, \neg walking\} & \sigma_4 &= \{alive, \neg loaded, \neg walking\} \\ \sigma_2 &= \{\neg alive, loaded, \neg walking\} & \sigma_5 &= \{\neg alive, \neg loaded, \neg walking\}.\end{aligned}$$

$$Res_0^-(SHOOT, \sigma_0) = Res^-(SHOOT, \sigma_0) = \{\sigma_5\}.$$

Since there is no fluent preserving statement wrt the action SHOOT and the fluent *walking*, we finally get

$$\boxed{Res(SHOOT, \sigma_0) = \{\sigma_5\}}.$$

Alternative solution

We can apply the reasoning method from $\mathcal{AQ}$ with modifications of domain descriptions. Specifically:

whenever constraints work for actions ramifications, the respective releases statements are to be added.

Alternative solution

We can apply the reasoning method from $\mathcal{AQ}$ with modifications of domain descriptions. Specifically:

whenever constraints work for actions ramifications, the respective releases statements are to be added.

Consider again the *Enticing Fred* example.

***always** walking $\rightarrow$ alive;*
***initially** loaded $\wedge$ walking*
*SHOOT **causes** $\neg$ loaded;*
*SHOOT **causes** $\neg$ alive **if** loaded;*
*SHOOT **releases** walking;*
*ENTICE **causes** walking.*

Example 2: Enticing Fred (cont.)

$$\sigma_0 = \{alive, loaded, walking\}$$

$$\sigma_1 = \{alive, \neg loaded, walking\}$$

$$\sigma_2 = \{\neg alive, loaded, walking\}$$

$$\sigma_3 = \{\neg alive, \neg loaded, walking\}$$

$$\sigma_4 = \{alive, loaded, \neg walking\}$$

$$\sigma_5 = \{alive, \neg loaded, \neg walking\}$$

$$\sigma_6 = \{\neg alive, loaded, \neg walking\}$$

$$\sigma_7 = \{\neg alive, \neg loaded, \neg walking\}$$

$$Res_0^-(SHOOT, \sigma_0) = \{\sigma_3, \sigma_7\}$$

$$* \quad New(SHOOT, \sigma_0, \sigma_3) = \{\neg alive, \neg loaded, walking\}$$

$$* \quad New(SHOOT, \sigma_0, \sigma_7) = \{\neg alive, \neg loaded, \neg walking\}$$

$$Res^-(SHOOT, \sigma_0) = \{\sigma_3, \sigma_7\}$$

$$\boxed{Res(SHOOT, \sigma_0) = \{\sigma_7\}.}$$

Thank you for your attention!

Any questions are welcome.